

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from tensorflow.keras.utils import to_categorical
from sklearn import datasets
```

```
iris = datasets.load_iris()
X = iris.data
target = iris.target
Y = to_categorical(target).astype("uint8")
# 0 => [1 0 0]
# 1 => [0 1 0]
# 2 => [0 0 1]
```

```
idx = 0
print(X[idx])
print(target[idx])
print(Y[idx])
```

```
[5.1 3.5 1.4 0.2]
0
[1 0 0]
```

```
model = keras.Sequential(
    [
        layers.Dense(3, activation="sigmoid", name="layer1", input_shape=(4,)),
        layers.Dense(3, activation="sigmoid", name="layer2"),
    ]
)

model.summary()
```

```
/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
layer1 (Dense)	(None, 3)	15
layer2 (Dense)	(None, 3)	12

```
Total params: 27 (108.00 B)
Trainable params: 27 (108.00 B)
Non-trainable params: 0 (0.00 B)
```

```
y = model(X)
print(y[149])

tf.Tensor([0.24113354 0.5489046 0.51716024], shape=(3,), dtype=float32)
```

```
optimizer=tf.keras.optimizers.SGD(learning_rate=0.03)
loss=tf.keras.losses.MeanSquaredError()

model.compile(optimizer, loss)
history = model.fit(X, Y, epochs=500)
```

```

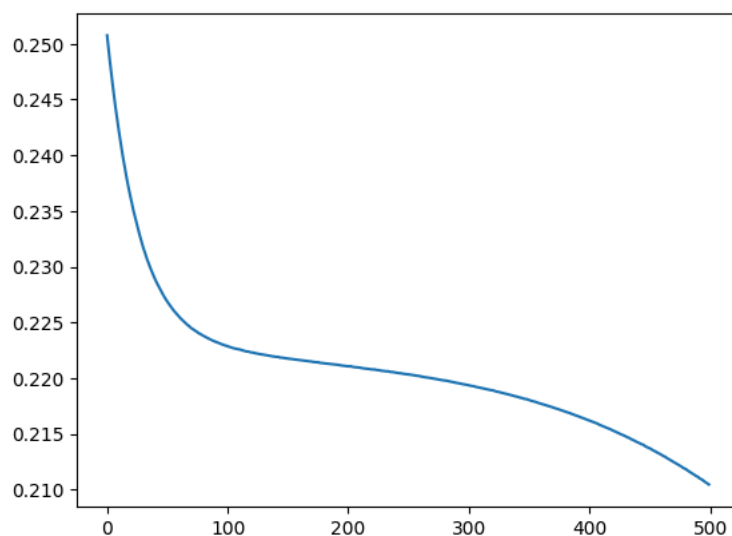
5/5 ----- 0s 8ms/step - loss: 0.2120
Epoch 479/500
5/5 ----- 0s 9ms/step - loss: 0.2119
Epoch 480/500
5/5 ----- 0s 15ms/step - loss: 0.2119
Epoch 481/500
5/5 ----- 0s 11ms/step - loss: 0.2118
Epoch 482/500
5/5 ----- 0s 8ms/step - loss: 0.2117
Epoch 483/500
5/5 ----- 0s 9ms/step - loss: 0.2117
Epoch 484/500
5/5 ----- 0s 10ms/step - loss: 0.2116
Epoch 485/500
5/5 ----- 0s 9ms/step - loss: 0.2115
Epoch 486/500
5/5 ----- 0s 9ms/step - loss: 0.2115
Epoch 487/500
5/5 ----- 0s 9ms/step - loss: 0.2114
Epoch 488/500
5/5 ----- 0s 9ms/step - loss: 0.2113
Epoch 489/500
5/5 ----- 0s 9ms/step - loss: 0.2113
Epoch 490/500
5/5 ----- 0s 10ms/step - loss: 0.2112
Epoch 491/500
5/5 ----- 0s 9ms/step - loss: 0.2111
Epoch 492/500
5/5 ----- 0s 8ms/step - loss: 0.2110
Epoch 493/500
5/5 ----- 0s 9ms/step - loss: 0.2110
Epoch 494/500
5/5 ----- 0s 11ms/step - loss: 0.2109
Epoch 495/500
5/5 ----- 0s 8ms/step - loss: 0.2109
Epoch 496/500
5/5 ----- 0s 8ms/step - loss: 0.2108
Epoch 497/500
5/5 ----- 0s 10ms/step - loss: 0.2107
Epoch 498/500
5/5 ----- 0s 9ms/step - loss: 0.2106
Epoch 499/500
5/5 ----- 0s 8ms/step - loss: 0.2106
Epoch 500/500
5/5 ----- 0s 10ms/step - loss: 0.2105

```

```

plt.plot(history.history['loss'])
plt.show()

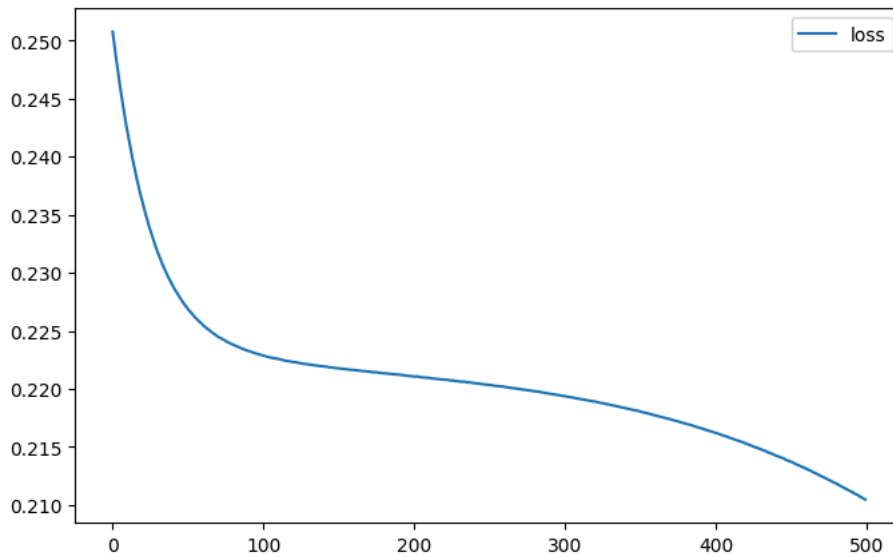
```



```

pd.DataFrame(history.history).plot(figsize=(8,5))
plt.show()

```



```
model.predict(np.array([[3,3,1,1]]))
```

```
1/1 — 0s 85ms/step
array([[0.35172293, 0.28259173, 0.285985  ]], dtype=float32)
```

Comienza a programar o [generar](#) con IA.

✓ Red profunda — topología $4 \times 3 \times 3 \times 3 \times 3$

Se añaden **dos capas** `layers.Dense(3, activation="sigmoid")` extra antes de la capa de salida (por eso hay 4 capas `Dense` en `model.summary()`). Se mantienen el mismo SGD con $\eta = 0.03$, MSE y **500 épocas** que en el modelo original.

```
model_deep = keras.Sequential(
    [
        layers.Dense(3, activation="sigmoid", name="layer1", input_shape=(4,)),
        layers.Dense(3, activation="sigmoid", name="layer2"),
        layers.Dense(3, activation="sigmoid", name="layer3"),
        layers.Dense(3, activation="sigmoid", name="layer4"),
    ]
)

model_deep.summary()
```

/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input\_shape`
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Model: "sequential_1"

Layer (type)	Output Shape	Param #
layer1 (Dense)	(None, 3)	15
layer2 (Dense)	(None, 3)	12
layer3 (Dense)	(None, 3)	12
layer4 (Dense)	(None, 3)	12

Total params: 51 (204.00 B)
Trainable params: 51 (204.00 B)
Non-trainable params: 0 (0.00 B)

```
optimizer_deep=tf.keras.optimizers.SGD(learning_rate=0.03)
loss_deep=tf.keras.losses.MeanSquaredError()

model_deep.compile(optimizer_deep, loss_deep)
history_deep = model_deep.fit(X, Y, epochs=500)
print('Loss final (500 epocas):', history_deep.history['loss'][-1])
```

```

Epoch 454/500
5/5 ————— 0s 10ms/step - loss: 0.2222
Epoch 455/500
5/5 ————— 0s 8ms/step - loss: 0.2222
Epoch 456/500
5/5 ————— 0s 8ms/step - loss: 0.2222
Epoch 457/500
5/5 ————— 0s 10ms/step - loss: 0.2222
Epoch 458/500
5/5 ————— 0s 10ms/step - loss: 0.2222
Epoch 459/500
5/5 ————— 0s 9ms/step - loss: 0.2222
Epoch 460/500
5/5 ————— 0s 14ms/step - loss: 0.2222
Epoch 461/500
5/5 ————— 0s 12ms/step - loss: 0.2222
Epoch 462/500
5/5 ————— 0s 8ms/step - loss: 0.2222
Epoch 463/500
5/5 ————— 0s 17ms/step - loss: 0.2222
Epoch 464/500
5/5 ————— 0s 9ms/step - loss: 0.2222
Epoch 465/500
5/5 ————— 0s 11ms/step - loss: 0.2222
Epoch 466/500
5/5 ————— 0s 11ms/step - loss: 0.2222
Epoch 467/500
5/5 ————— 0s 9ms/step - loss: 0.2222
Epoch 468/500
5/5 ————— 0s 10ms/step - loss: 0.2222
Epoch 469/500
5/5 ————— 0s 10ms/step - loss: 0.2222
Epoch 470/500
5/5 ————— 0s 12ms/step - loss: 0.2222
Epoch 471/500
5/5 ————— 0s 11ms/step - loss: 0.2222
Epoch 472/500
5/5 ————— 0s 12ms/step - loss: 0.2222
Epoch 473/500
5/5 ————— 0s 10ms/step - loss: 0.2222
Epoch 474/500
5/5 ————— 0s 11ms/step - loss: 0.2222
Epoch 475/500
5/5 ————— 0s 9ms/step - loss: 0.2222
Epoch 476/500
5/5 ————— 0s 11ms/step - loss: 0.2222
Epoch 477/500
5/5 ————— 0s 12ms/step - loss: 0.2222
Epoch 478/500
5/5 ————— 0s 9ms/step - loss: 0.2222
Epoch 479/500
5/5 ————— 0s 9ms/step - loss: 0.2222
Epoch 480/500
5/5 ————— 0s 15ms/step - loss: 0.2222
Epoch 481/500
5/5 ————— 0s 9ms/step - loss: 0.2222
Epoch 482/500
5/5 ————— 0s 9ms/step - loss: 0.2222
Epoch 483/500

```

```

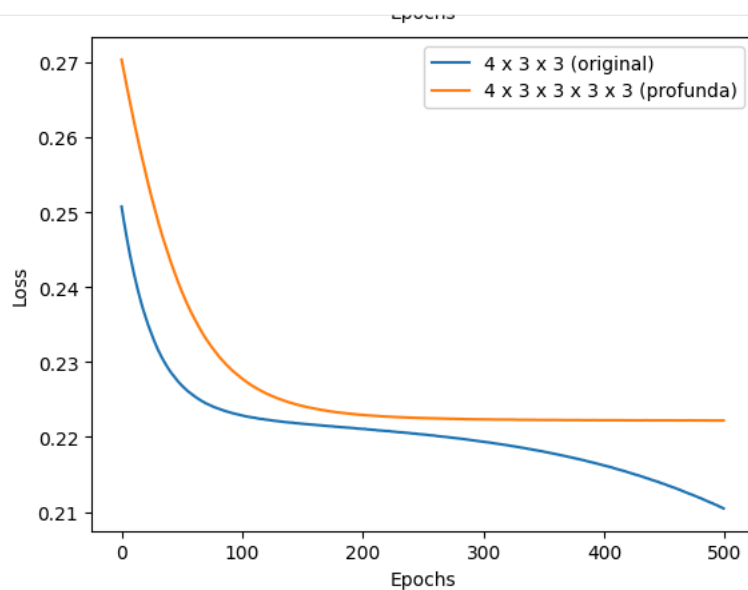
plt.plot(history_deep.history['loss'])
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.show()

```

Comparación de curvas

Pérdida (loss) por época del modelo original $4 \times 3 \times 3$ y del modelo profundo $4 \times 3 \times 3 \times 3 \times 3$ (se requiere haber corrido antes la celda de entrenamiento original).

```
plt.plot(history.history['loss'], label='4 x 3 x 3 (original)')
plt.plot(history_deep.history['loss'], label='4 x 3 x 3 x 3 x 3 (profunda)')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()
```



```
model_deep.predict(np.array([[3,3,1,1]]))
```

```
1/1 — 0s 98ms/step
array([[0.33530888, 0.3338049 , 0.33539835]], dtype=float32)
```